Compte rendu TP4 Deep Learning
Réalisé par Meriam Boukraa

Exercice 1:

Etape 1:

Voici le code à compléter:

```python
import os
import json
import time
import random
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
from torch.utils.tensorboard import SummaryWriter
from tqdm import tqdm

# Reproductibilité
torch.manual_seed(42)
random.seed(42)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
DATA_DIR = "/mnt/ssd/irishman"


def load_data(file_path):
    with open(file_path, "r", encoding="utf-8") as f:
        return json.load(f)


train_data = load_data(os.path.join(DATA_DIR, "train.json"))
val_data = load_data(os.path.join(DATA_DIR, "validation.json"))

print(f"Nombre de chansons (Train) : {len(train_data)}")
print(f"Nombre de chansons (Val)   : {len(val_data)}")

print("\n--- Exemple de partition (Train[0]) ---")
print(train_data[0]["abc notation"])
```

Exécution du script:

```
[controller] mboukraa@arcadia-slurm-controller:~/TP-HPC (main)
$ python TP4/train_music.py
Nombre de chansons (Train) : 214122
Nombre de chansons (Val)   : 2162

--- Exemple de partition (Train[0]) ---
X:1
L:1/8
M:4/4
K:Emin
|: E2 EF E2 EF | DEFG AFDF | E2 EF E2 B2 |1 efe^d e2 e2 :|2 efe^d e3 B |: e2
 ef g2 fe |
 defg afdf |1 e2 ef g2 fe | efe^d e3 B :|2 g2 bg f2 af | efe^d e2 e2 ||
[controller] mboukraa@arcadia-slurm-controller:~/TP-HPC (main)
$
```
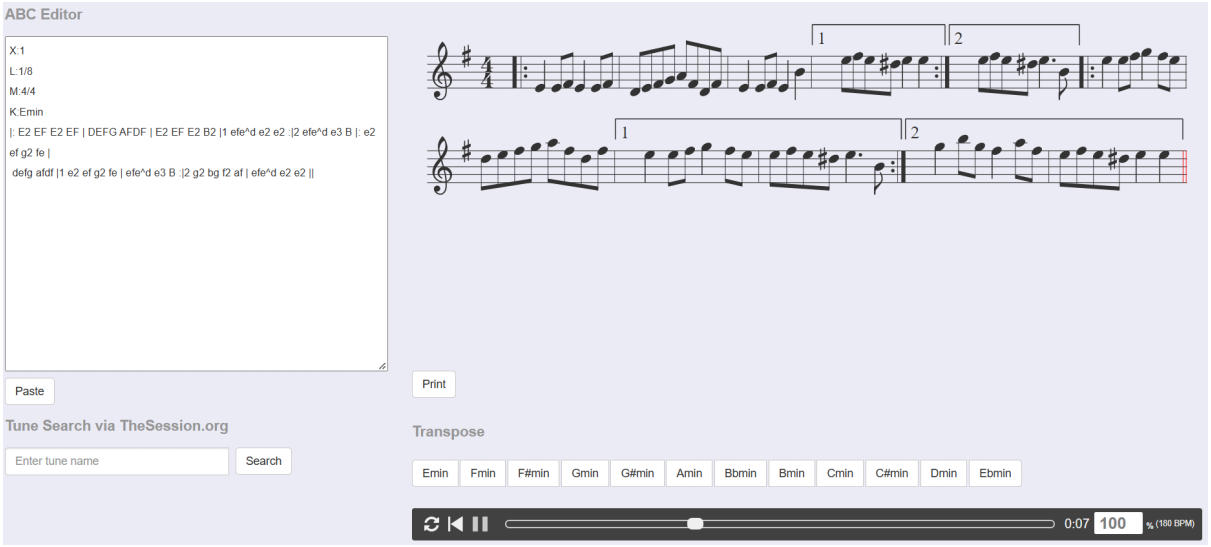
Réponse aux questions:

1. Les résultats obtenus sont les suivants :

- Nombre de partitions d'entraînement : 214 122.
- Nombre de partitions de validation : 2 162.

2.

J'ai bien collé le texte de la première partition dans ABC Player et je l'ai écouté. Voici le résultat:



Rôle des métadonnées:

Les premières lignes contiennent les métadonnées nécessaires à l'interprétation musicale de la partition :

- X:1 : identifiant de la partition.
- L:1/8 : durée par défaut d'une note, correspondant à une croche.
- M:4/4 : indique une métrique à quatre temps.
- K:Emin : indique la tonalité de mi mineur (E minor).

Les lignes suivantes décrivent la mélodie. Les lettres représentent les notes, les chiffres qui les suivent modifient leur durée, et les barres verticales délimitent les mesures. Les symboles de répétition permettent de rejouer certaines parties de la mélodie. Le symbole ^ indique une altération dièse : ^d représente donc un ré dièse.

Etape 2:

Voici le code à compléter:

```
# 1. Extraction déterministe des caractères uniques
all_text = "".join(
    song["abc notation"] for song in train_data + val_data
)

unique_chars = sorted(set(all_text))
vocab_size = len(unique_chars)

print(f"\nTaille du vocabulaire : {vocab_size} caractères uniques")

# 2. Mappings caractère <-> index
char_to_index = {
    char: idx for idx, char in enumerate(unique_chars)
}

index_to_char = {
    idx: char for idx, char in enumerate(unique_chars)
}


def vectorize_string(s):
    return [char_to_index[char] for char in s]


# 3. Statistiques de longueur et Padding
lengths = [len(song["abc notation"]) for song in train_data]

max_len = max(lengths)
avg_len = sum(lengths) / len(lengths)

print(f"Longueur moyenne : {avg_len:.1f} | Longueur max : {max_len}")
```

```
SEQ_LENGTH = 500


def pad_or_truncate(sequence, max_length=SEQ_LENGTH):
    if len(sequence) < max_length:
        return sequence + " " * (max_length - len(sequence))
    else:
        return sequence[:max_length]
```

Résultat de l’exécution:

```
[controller] mboukraa@arcadia-slurm-controller:~/TP-HPC (main)
$ python TP4/train_music.py
Nombre de chansons (Train) : 214122
Nombre de chansons (Val)   : 2162

--- Exemple de partition (Train[0]) ---
X:1
L:1/8
M:4/4
K:Emin
|: E2 EF E2 EF | DEFG AFDF | E2 EF E2 B2 |1 efe^d e2 e2 :|2 efe^d e3 B |: e2
 ef g2 fe |
 defg afdf |1 e2 ef g2 fe | efe^d e3 B :|2 g2 bg f2 af | efe^d e2 e2 ||

Taille du vocabulaire : 95 caractères uniques
Longueur moyenne : 290.1 | Longueur max : 2968

Longueur de l'exemple original : 183
Longueur après padding/troncature : 500
Longueur de la version vectorisée : 183
Index du caractère 'A' : 33
[controller] mboukraa@arcadia-slurm-controller:~/TP-HPC (main)
$
```

Réponse aux questions:

1.

Les statistiques obtenues sur le jeu d'entraînement sont les suivantes :

- Taille du vocabulaire : 95 caractères uniques.
- Longueur moyenne d'une partition : 290,1 caractères.
- Longueur maximale d'une partition : 2 968 caractères.
- Longueur fixe choisie : 500 caractères.

2.

Sans sorted(), l'ordre des caractères pourrait changer d'une exécution à l'autre. Ainsi, un même indice pourrait correspondre à un caractère différent. Même si les poids du modèle sont correctement sauvegardés, le modèle interpréterait mal les caractères lors de la génération et pourrait produire une musique incohérente. L'utilisation de sorted() garantit donc un ordre stable du vocabulaire, qu'il faut également sauvegarder avec le modèle pour conserver la correspondance entre les caractères et leurs indices.

3.

Nous avons choisi SEQ_LENGTH = 500 pour limiter le coût des calculs tout en conservant une partie importante des partitions, dont la longueur moyenne est de 290,1 caractères.

Aligner toutes les chansons sur max_len = 2968 augmenterait le temps de calcul et la mémoire GPU nécessaires, car le modèle traiterait beaucoup de caractères de padding inutiles. De plus, si ces caractères étaient pris en compte dans la fonction de perte, ils pourraient perturber l'apprentissage en incitant le modèle à prédire trop d'espaces. Enfin, la

troncature à 500 caractères entraîne la perte d'une partie des chansons les plus longues, ce qui constitue un compromis entre la qualité des données et le coût du calcul.

Exercice 2:

Etape 1:

Voici le code de la classe MusicDataset à compléter:

```
class MusicDataset(Dataset):
    def __init__(self, raw_data, seq_length=SEQ_LENGTH, subset_size=None):
        if subset_size is not None:
            raw_data = raw_data[:subset_size]

        self.data = [
            vectorize_string(
                pad_or_truncate(song["abc notation"], seq_length)
            )
            for song in raw_data
        ]

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        seq = self.data[idx]

        # Entrée : tous les caractères sauf le dernier
        x = torch.tensor(seq[:-1], dtype=torch.long)

        # Cible : tous les caractères sauf le premier
        y = torch.tensor(seq[1:], dtype=torch.long)

        return x, y
```

Vérification de MusicDataset:

```
# Vérification du MusicDataset
train_dataset = MusicDataset(
    train_data, seq_length=SEQ_LENGTH, subset_size=100
)
train_loader = DataLoader(train_dataset, batch_size=128, shuffle=True)

x, y = next(iter(train_loader))

print("\n--- Vérification MusicDataset ---")
print("Taille du dataset :", len(train_dataset))
print("Shape de x :", x.shape)
print("Shape de y :", y.shape)
print("Type de x :", x.dtype)
print("Type de y :", y.dtype)
```

Résultat de l'exécution:

```
--- Vérification MusicDataset ---
Taille du dataset : 100
Shape de x : torch.Size([100, 499])
Shape de y : torch.Size([100, 499])
Type de x : torch.int64
Type de y : torch.int64
[controller] mboukraa@arcadia-slurm-controller:~/TP-HPC (main)
$
```

Réponse à la question:

Avec SEQ_LENGTH = 500, les séquences d'entrée et les cibles contiennent chacune 499 caractères, car on décale la séquence d'un caractère pour apprendre à prédire le suivant.

Avec BATCH_SIZE = 128, les dimensions des tenseurs sont donc :

- Entrée x : (128, 499).
- Cible y : (128, 499).

Le type torch.long est utilisé car chaque caractère est représenté par un indice entier du vocabulaire. Ce type est requis par la couche nn.Embedding, qui utilise ces indices pour sélectionner les vecteurs correspondants. Contrairement à torch.float32, il représente directement les indices entiers attendus par cette couche.

Etape 2:

Voici le code à compléter:

```python
class MusicModel(nn.Module):
    def __init__(self, vocab_size, embedding_dim=256, hidden_size=512, rnn_>
        super().__init__()

        self.rnn_type = rnn_type.lower()

        # 1. Couche d'embedding
        self.embedding = nn.Embedding(vocab_size, embedding_dim)

        # 2. Couche récurrente
        if self.rnn_type == "lstm":
            self.rnn = nn.LSTM(
                embedding_dim, hidden_size, batch_first=True
            )
        elif self.rnn_type == "rnn":
            self.rnn = nn.RNN(
                embedding_dim, hidden_size, batch_first=True
            )
        else:
            raise ValueError("rnn_type doit être 'rnn' ou 'lstm'")

        # 3. Couche de sortie
        self.fc = nn.Linear(hidden_size, vocab_size)

    def forward(self, x, hidden_state=None):
        embeds = self.embedding(x)
        out, hidden_state = self.rnn(embeds, hidden_state)
        logits = self.fc(out)
        return logits, hidden_state


def count_parameters(model):
    return sum(
        p.numel() for p in model.parameters() if p.requires_grad
    )
```

Réponse aux questions:

1.

On utilise les valeurs suivantes : vocab_size = 95, embedding_dim = 256 et hidden_size = 512.

- Modèle RNN (rnn_type="rnn")
- Couche Embedding

Chaque caractère possède un vecteur de 256 valeurs. Pour 95 caractères :

Nombre de paramètres = 95 × 256 = 24 320

- Couche récurrente RNN

La couche possède deux matrices de poids et deux vecteurs de biais :

- Poids entrée → caché : 512 × 256 = 131 072
- Poids caché → caché : 512 × 512 = 262 144
- Premier biais : 512
- Deuxième biais : 512

Total RNN = 131 072 + 262 144 + 512 + 512 = 394 240

- Couche linéaire fc

Elle transforme les 512 valeurs cachées en 95 scores, avec 95 biais.

Nombre de paramètres = (512 × 95) + 95 = 48 735

Total du modèle RNN :

24 320 + 394 240 + 48 735 = 467 295 paramètres

- Modèle LSTM (rnn_type="lstm")
- Couche Embedding

Identique à celle du RNN : 24 320 paramètres.

- Couche récurrente LSTM

Le LSTM possède quatre portes. Chaque porte possède le même nombre de paramètres que la couche récurrente RNN.

Nombre de paramètres = 4 × 394 240 = 1 576 960

- Couche linéaire fc

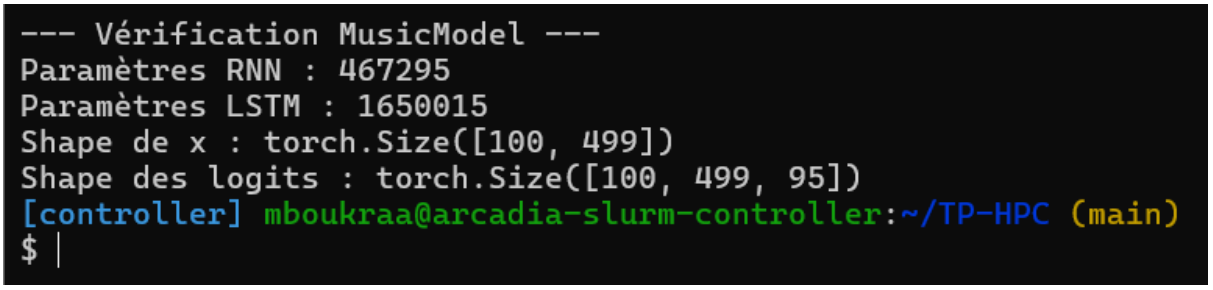
Identique à celle du RNN : 48 735 paramètres.

Total du modèle LSTM :

24 320 + 1 576 960 + 48 735 = 1 650 015 paramètres

Vérification avec count_parameters()

Les résultats obtenus avec le programme correspondent exactement aux calculs théoriques:



Pourquoi le LSTM possède-t-il quatre fois plus de paramètres que le RNN ?

Le RNN classique utilise un seul calcul récurrent, tandis que le LSTM possède quatre portes qui contrôlent les informations à conserver, à oublier et à transmettre. Chaque porte possède ses propres poids et ses deux vecteurs de biais. La couche récurrente du LSTM possède donc exactement quatre fois plus de paramètres que celle du RNN.

2. La couche nn.Embedding associe à chaque caractère un vecteur de valeurs appris pendant l'entraînement. Elle traite les indices comme des identifiants, sans leur attribuer de relation numérique artificielle.

À l'inverse, une couche linéaire appliquée directement à un indice scalaire pourrait introduire une relation numérique artificielle entre les caractères. L'embedding permet au modèle d'apprendre une représentation adaptée à chaque caractère, ce qui est plus pertinent pour la génération musicale.

3. Avec nn.RNN, la variable hidden_state contient un seul état caché, qui résume les informations retenues depuis les caractères précédents.

Avec nn.LSTM, hidden_state est un tuple contenant deux états :

- h_n (état caché) : représente l'information transmise à la sortie du réseau et utilisée pour les prédictions.
- c_n (état de cellule) : conserve une mémoire interne permettant de retenir des informations sur une plus longue durée et de limiter le problème de disparition du gradient.

Ainsi, contrairement au RNN classique, le LSTM dispose d'une mémoire interne supplémentaire qui l'aide à apprendre les dépendances à long terme dans les séquences musicales.

Exercice 3:

Réponses aux questions:

1.

Manipulation des tensors:

La fonction CrossEntropyLoss() attend des prédictions sous la forme (N, vocab_size) et des cibles sous la forme (N). On utilise donc reshape() pour transformer les tenseurs et regrouper toutes les positions des séquences.

Le paramètre -1 permet à PyTorch de calculer automatiquement la dimension manquante. Avec un batch de 128 séquences de longueur 499, on obtient :

- logits : (63872, 95)
- y : (63872,)

Car $128 \times 499 = 63872$.

2.

loss.backward() calcule les gradients, puis clip_grad_norm_() limite leur norme s'ils sont trop élevés. Enfin, optimizer.step() met à jour les paramètres du modèle.

Le clipping doit donc être effectué après le calcul des gradients et avant la mise à jour des paramètres. Il permet d'éviter les gradients explosifs et de stabiliser l'entraînement.

3.

Entrainement de RNN sans clipping pendant 15 époques:

```
[rnn_sub10000_lr0.005_noclip_110104] Epoch 15/15:  87%|          | 69/79 [00
[rnn_sub10000_lr0.005_noclip_110104] Epoch 15/15:  92%|          | 73/79 [00
[rnn_sub10000_lr0.005_noclip_110104] Epoch 15/15:  97%|          | 77/79 [00

Epoch 15 | Train Loss: 0.7768 | Val Loss: 0.7761 | Val Acc: 0.7594 | Avg Gra
dNorm: 0.255
Expérience terminée : rnn_sub10000_lr0.005_noclip_110104
Meilleure Val Loss : 0.7291
[compute] [job 14289] mboukraa@starfighter-slurm-node-05-1:~/TP-HPC (main)
$
```

Entrainement de RNN avec clipping pendant 15 époques:

```
[rnn_sub10000_lr0.005_clip1.0_105644] Epoch 15/15:  87%|          | 69/79 [0
[rnn_sub10000_lr0.005_clip1.0_105644] Epoch 15/15:  92%|          | 73/79 [0
[rnn_sub10000_lr0.005_clip1.0_105644] Epoch 15/15:  97%|          | 77/79 [0

Epoch 15 | Train Loss: 0.7153 | Val Loss: 0.7391 | Val Acc: 0.7695 | Avg Gra
dNorm: 0.818
Expérience terminée : rnn_sub10000_lr0.005_clip1.0_105644
Meilleure Val Loss : 0.7176
[compute] [job 14289] mboukraa@starfighter-slurm-node-05-1:~/TP-HPC (main)
```

Entrainement de LSTM avec clipping pendant 15 époques:

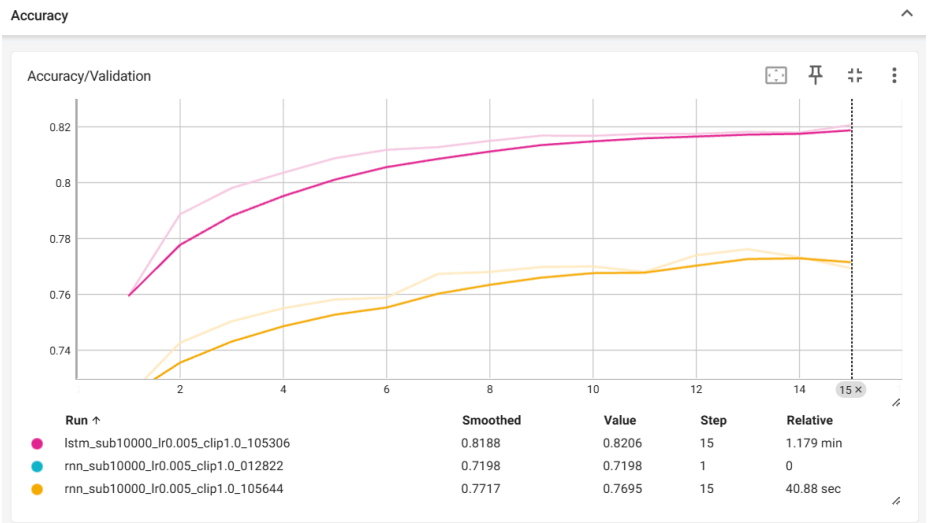
```
[lstm_sub10000_lr0.005_clip1.0_105306] Epoch 15/15:  87%|          | 69/79 [
[lstm_sub10000_lr0.005_clip1.0_105306] Epoch 15/15:  90%|          | 71/79 [
[lstm_sub10000_lr0.005_clip1.0_105306] Epoch 15/15:  92%|          | 73/79 [
[lstm_sub10000_lr0.005_clip1.0_105306] Epoch 15/15:  95%|          | 75/79 [
[lstm_sub10000_lr0.005_clip1.0_105306] Epoch 15/15:  97%|          | 77/79 [

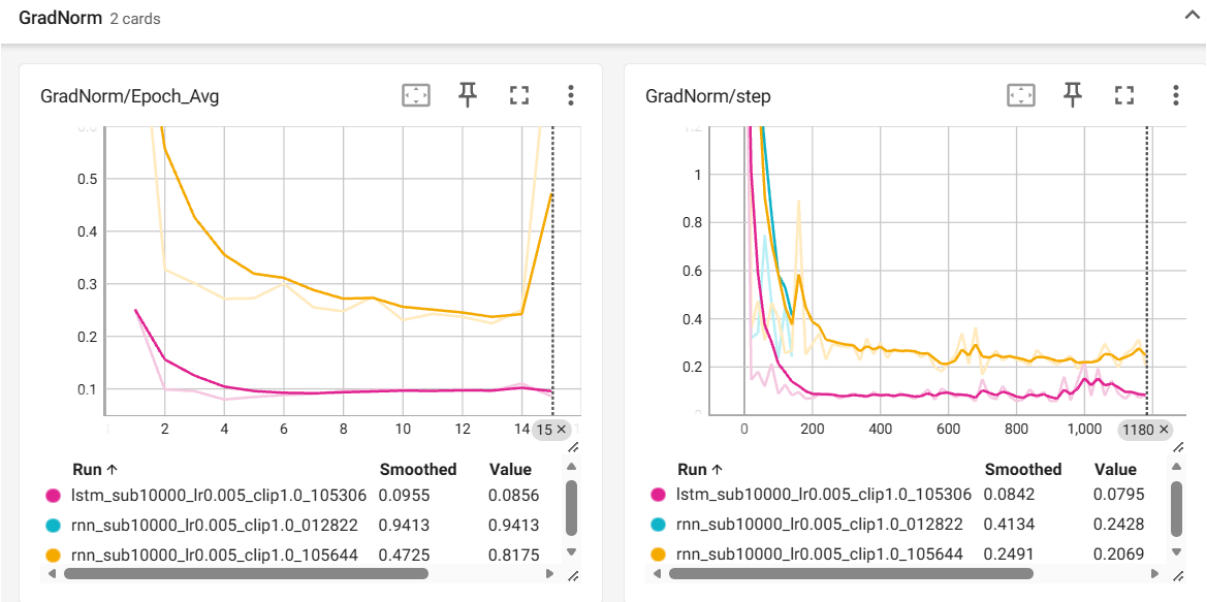
Epoch 15 | Train Loss: 0.4919 | Val Loss: 0.5584 | Val Acc: 0.8206 | Avg Gra
dNorm: 0.086
Meilleur modèle sauvegardé : best_lstm_clip1.0.pth
Expérience terminée : lstm_sub10000_lr0.005_clip1.0_105306
Meilleure Val Loss : 0.5584
[compute] [job 14289] mboukraa@starfighter-slurm-node-05-1:~/TP-HPC (main)
```

Interprétation:

- Le LSTM est le meilleur des trois modèles sur ces résultats : il obtient la plus faible perte de validation (0,5584) et la meilleure précision (82,06 %).
- Le gradient clipping améliore légèrement le meilleur résultat du RNN : la Val Loss passe de 0,7291 à 0,7176. La différence reste faible.
- Le LSTM est plus complexe, avec environ 3,5 fois plus de paramètres que le RNN, mais ses performances sont meilleures sur cette tâche.
- Le clipping limite la norme des gradients, mais l'Avg GradNorm affichée ne suffit pas à elle seule pour conclure qu'un modèle est plus stable : il faudrait aussi examiner son évolution pendant l'entraînement.

Les courbes Tensorboard:





Analyse expérimentale (TensorBoard):

- Run 1 vs Run 2 : Effet du gradient clipping

Le gradient clipping stabilise l'entraînement du RNN en limitant la norme des gradients trop élevés. Il permet de réduire le risque d'explosion des gradients, un problème du RNN simple lié à la rétropropagation à travers le temps. L'accuracy de validation passe d'environ 75,94 % à 76,95 % dans nos résultats précédents, avec une amélioration de la loss de validation.

- Run 2 vs Run 3 : RNN vs LSTM

Le LSTM obtient de meilleures performances : son accuracy de validation atteint 82,06 %, contre 76,95 % pour le RNN, et sa loss de validation finale est plus faible (0,5584 contre 0,7391). Il est cependant plus lent à entraîner (environ 1,18 min contre 40,88 s). Les courbes permettent de comparer la vitesse de convergence.

4.

Lorsqu'une partition de 250 caractères est complétée jusqu'à 500 caractères avec des espaces, la moitié des positions correspond au padding. Le modèle peut donc obtenir une accuracy élevée en prédisant correctement ces espaces, même s'il prédit mal les notes musicales.

Pour ignorer le padding dans le calcul de la perte, on peut utiliser :

loss_fn = nn.CrossEntropyLoss(ignore_index=pad_index)

Cela exclut les positions de padding de la perte. Il faut toutefois utiliser un token spécial <PAD> avec son propre indice, car les espaces peuvent aussi apparaître dans les véritables partitions musicales.

Exercice 4:

Etape 1:

Voici le code de la fonction generate_music à compléter:

```python
@torch.no_grad()
def generate_music(model, start_sequence, output_length=300, temperature=1.>
    model.eval()
    generated_seq = start_sequence

    input_indices = [char_to_index[c] for c in start_sequence]
    input_tensor = torch.tensor(
        input_indices, dtype=torch.long, device=device
    ).unsqueeze(0)

    logits, hidden_state = model(input_tensor, hidden_state=None)

    for _ in range(output_length):
        last_logits = logits[:, -1, :] / temperature
        probs = F.softmax(last_logits, dim=-1).squeeze(0)
        next_index = torch.multinomial(probs, num_samples=1).item()
        next_char = index_to_char[next_index]

        generated_seq += next_char
        input_tensor = torch.tensor(
            [[next_index]], dtype=torch.long, device=device
        )
        logits, hidden_state = model(input_tensor, hidden_state=hidden_stat>

    return generated_seq
```

Les trois partitions ont été bien enregistrées:

```
Partition enregistrée : TP4/generated/irish_reel_T0.2.abc
Partition enregistrée : TP4/generated/irish_reel_T0.8.abc
Partition enregistrée : TP4/generated/irish_reel_T2.0.abc
Terminé ! Les trois partitions sont dans TP4/generated/
[controller] mboukraa@arcadia-slurm-controller:~/TP-HPC (main)
```

Réponses aux questions:

1. Efficacité algorithmique :

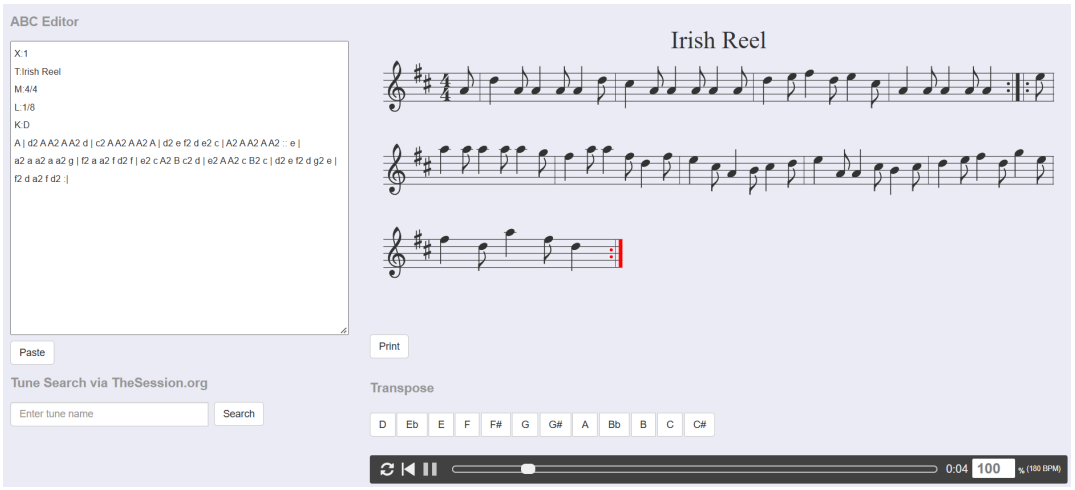
- Sans conserver hidden_state, la complexité temporelle est $O(N^2)$, car toute la séquence est recalculée à chaque itération.
- Avec hidden_state, elle est $O(N)$, car seul le dernier caractère est traité à chaque itération.

2. Rôle mathématique de la température T :

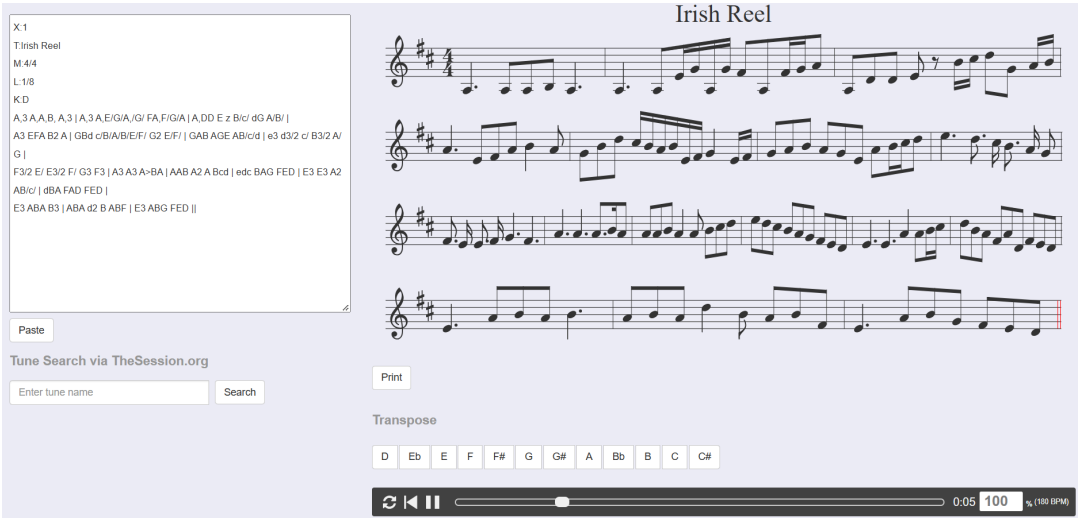
- Lorsque $T \to 0^+$, la distribution se concentre sur le caractère ayant le logit maximal. Cela correspond à un décodage glouton (greedy decoding).
- Lorsque $T \to +\infty$, la distribution tend vers une distribution uniforme sur tous les caractères du vocabulaire : le choix devient aléatoire.

3.

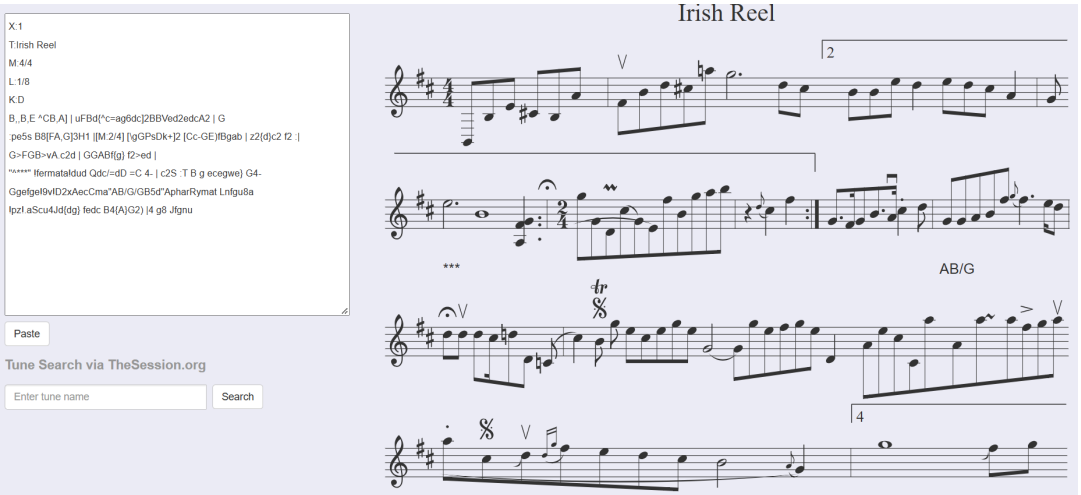
La partition irish_reel_T0.2.abc :



La partition irish_reel_T0.8.abc :



La partition irish_reel_T2.0.abc :



Analyse de la validité syntaxique et de la richesse musicale:

Température 0.2 :

La partition présente une structure relativement régulière, avec des motifs répétitifs et des notes principalement cohérentes avec la tonalité de ré majeur (K:D). Les barres de mesure et les symboles ABC semblent globalement bien utilisés. Cependant, la forte répétition des mêmes notes et motifs limite la richesse musicale et rend la mélodie moins variée.

Température 0.8 :

La génération présente davantage de diversité dans les notes, les rythmes et les motifs musicaux. La partition utilise plusieurs notations ABC valides, notamment les altérations, les durées fractionnaires et les silences. Elle semble offrir un meilleur compromis entre répétition et diversité, même si la validité de toutes les mesures et la cohérence musicale doivent être confirmées par la lecture et l'écoute.

Température 2.0 :

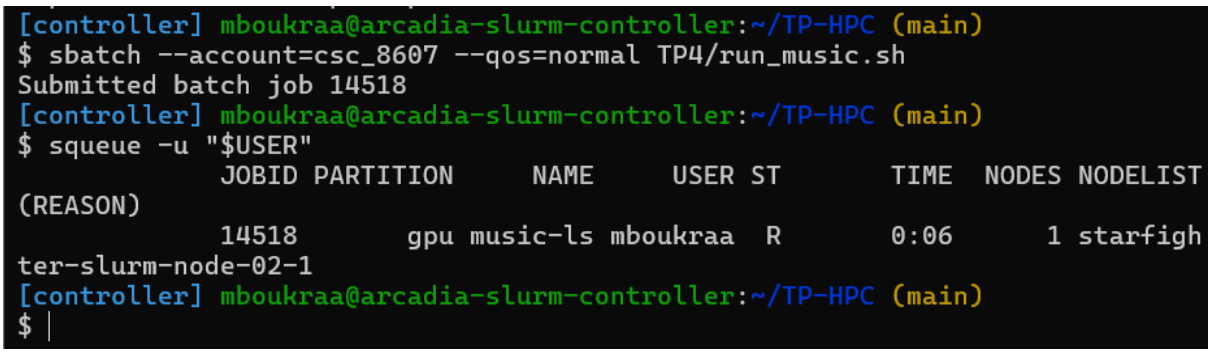
La partition contient une grande variété de caractères et de symboles, mais plusieurs séquences paraissent syntaxiquement incorrectes ou inhabituelles pour la notation ABC. La structure musicale est moins lisible et les motifs semblent plus chaotiques. Cette génération illustre les limites d'une température élevée : la diversité augmente, mais la cohérence syntaxique et musicale peut fortement diminuer.

Conclusion :

La température influence directement le compromis entre prévisibilité et diversité. Une température faible produit une génération plus répétitive, tandis qu'une température élevée favorise des séquences plus aléatoires, parfois invalides. Dans nos résultats, la température 0.8 semble offrir le meilleur compromis, sous réserve de vérifier la validité des mesures et de l'écoute des trois partitions.

Etape 2:

Lancement d'un job Slurm:



On peut voir que le job est entrain de tourner:

```
[controller] mboukraa@arcadia-slurm-controller:~/TP-HPC (main)
$ tail -f logs/music-lstm-14518.out
Epoch 05 | Train Loss: 0.5377 | Val Loss: 0.5380 | Val Acc: 0.8236 | Avg Gra
dNorm: 0.160
Meilleur modèle sauvegardé : best_lstm_clip1.0.pth
Epoch 06 | Train Loss: 0.5248 | Val Loss: 0.5259 | Val Acc: 0.8278 | Avg Gra
dNorm: 0.142
Meilleur modèle sauvegardé : best_lstm_clip1.0.pth
Epoch 07 | Train Loss: 0.5126 | Val Loss: 0.5179 | Val Acc: 0.8299 | Avg Gra
dNorm: 0.133
Meilleur modèle sauvegardé : best_lstm_clip1.0.pth
Epoch 08 | Train Loss: 0.5017 | Val Loss: 0.5117 | Val Acc: 0.8326 | Avg Gra
dNorm: 0.127
Meilleur modèle sauvegardé : best_lstm_clip1.0.pth
Epoch 09 | Train Loss: 0.4935 | Val Loss: 0.5069 | Val Acc: 0.8335 | Avg Gra
dNorm: 0.123
```

A la fin, on peut bien voir que l'entraînement du LSTM s'est terminé avec succès. Les trois fichiers ABC correspondant aux températures 0.2, 0.8 et 2.0 ont bien été créés. Ils ont chacun une taille de 333 octets, ce qui confirme leur génération, mais pas nécessairement la validité musicale de leur contenu:

```
[controller] mboukraa@arcadia-slurm-controller:~/TP-HPC (main)
$ sacct -j 14518 --format=JobID,State,ExitCode,Elapsed
JobID             State ExitCode    Elapsed
------------ ---------- -------- ----------
14518         COMPLETED      0:0   00:27:21
14518.batch   COMPLETED      0:0   00:27:21
14518.extern  COMPLETED      0:0   00:27:22
[controller] mboukraa@arcadia-slurm-controller:~/TP-HPC (main)
$ ls -lh TP4/generated/irish_reel_large_T*.abc
.rw-r--r-- mboukraa 1617 333 B Wed Oct 28 14:39:17 2026 ◆ TP4/generated/iris
h_reel_large_T0.2.abc
.rw-r--r-- mboukraa 1617 333 B Wed Oct 28 14:39:17 2026 ◆ TP4/generated/iris
h_reel_large_T0.8.abc
.rw-r--r-- mboukraa 1617 333 B Wed Oct 28 14:39:18 2026 ◆ TP4/generated/iris
h_reel_large_T2.0.abc
[controller] mboukraa@arcadia-slurm-controller:~/TP-HPC (main)
$
```

Réponse à la question 4c:

La transposition consiste à décaler toutes les notes d'une mélodie de quelques demi-tons pour créer une nouvelle version dans une autre tonalité. En format ABC, il faudrait modifier les notes et l'en-tête K : tout en conservant le rythme et la structure de la mélodie.

Cette technique permettrait d'augmenter la diversité des données d'entraînement sans avoir besoin de nouvelles mélodies. Le modèle apprendrait ainsi à reconnaître les structures musicales plutôt qu'à mémoriser les exemples, ce qui réduirait le sur-apprentissage.

L’entraînement du LSTM avec 5 millions de paramètres a duré environ 27 minutes et s’est terminé avec succès après 30 epochs. La précision de validation atteint environ 84 %, avec une perte de validation minimale de 0,4807 à l’epoch 20. La perte augmente ensuite globalement, ce qui peut indiquer un début de sur-apprentissage:

```
[controller] mboukraa@arcadia-slurm-controller:~/TP-HPC (main)
$ grep -iE "epoch|early stopping|entraînement terminé" logs/music-lstm-14518
.out | tail -n 15
Epoch 16 | Train Loss: 0.4487 | Val Loss: 0.4849 | Val Acc: 0.8422 | Avg Gra
dNorm: 0.090
Epoch 17 | Train Loss: 0.4452 | Val Loss: 0.4855 | Val Acc: 0.8419 | Avg Gra
dNorm: 0.083
Epoch 18 | Train Loss: 0.4473 | Val Loss: 0.4882 | Val Acc: 0.8404 | Avg Gra
dNorm: 0.083
Epoch 19 | Train Loss: 0.4477 | Val Loss: 0.4839 | Val Acc: 0.8419 | Avg Gra
dNorm: 0.083
Epoch 20 | Train Loss: 0.4393 | Val Loss: 0.4807 | Val Acc: 0.8432 | Avg Gra
dNorm: 0.079
Epoch 21 | Train Loss: 0.4375 | Val Loss: 0.4834 | Val Acc: 0.8429 | Avg Gra
dNorm: 0.081
Epoch 22 | Train Loss: 0.4421 | Val Loss: 0.4873 | Val Acc: 0.8412 | Avg Gra
dNorm: 0.081
Epoch 23 | Train Loss: 0.4474 | Val Loss: 0.4887 | Val Acc: 0.8411 | Avg Gra
dNorm: 0.080
Epoch 24 | Train Loss: 0.4454 | Val Loss: 0.4851 | Val Acc: 0.8419 | Avg Gra
dNorm: 0.072
Epoch 25 | Train Loss: 0.4359 | Val Loss: 0.4841 | Val Acc: 0.8419 | Avg Gra
dNorm: 0.077
Epoch 26 | Train Loss: 0.5003 | Val Loss: 0.5027 | Val Acc: 0.8359 | Avg Gra
dNorm: 0.235
Epoch 27 | Train Loss: 0.4734 | Val Loss: 0.4922 | Val Acc: 0.8401 | Avg Gra
dNorm: 0.071
Epoch 28 | Train Loss: 0.4622 | Val Loss: 0.4901 | Val Acc: 0.8403 | Avg Gra
dNorm: 0.074
Epoch 29 | Train Loss: 0.4659 | Val Loss: 0.4892 | Val Acc: 0.8403 | Avg Gra
dNorm: 0.079
Epoch 30 | Train Loss: 0.4823 | Val Loss: 0.5058 | Val Acc: 0.8355 | Avg Gra
dNorm: 0.568
[controller] mboukraa@arcadia-slurm-controller:~/TP-HPC (main)
$
```

La comparaison des trois températures montre que T = 0.2 produit une mélodie simple et répétitive, tandis que T = 0.8 génère une séquence plus variée et globalement cohérente. En revanche, T = 2.0 produit beaucoup de caractères et de structures inhabituels, rendant la partition difficile à interpréter. Une température intermédiaire semble donc offrir un meilleur compromis entre diversité et cohérence musicale:

```
[controller] mboukraa@arcadia-slurm-controller:~/TP-HPC (main)
$ for f in TP4/generated/irish_reel_large_T*.abc; do
    echo "===== $f ====="
    cat "$f"
    echo
done
===== TP4/generated/irish_reel_large_T0.2.abc =====
X:1
T:Irish Reel
M:4/4
L:1/8
K:D
 A | d2 d d d e | f2 d d e d | c2 A A A A | d2 d c B A | d2 d d e f | e d c
B A ||


===== TP4/generated/irish_reel_large_T0.8.abc =====
X:1
T:Irish Reel
M:4/4
L:1/8
K:D
 A3/2 F/ A d d A | A/ A3/2 B A A | d3/2 e/ d f d d | B A/^G/ A3 A | f d d c/
B/ c e |
 d f/g/ a f e d | f2 e2 d/e/d/B/ | A F/F/ F A/A/ A d | f e/f/ g e/d/ c B | A
2 A/G/F/E/ D3 |
 A/B/c/d/ e/f/ g a/g/f/e/ | e f/g/ a/f/ e A ||

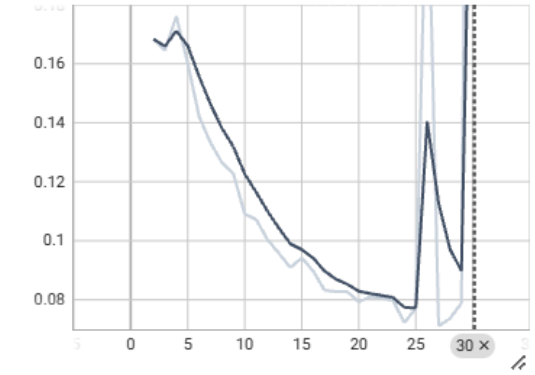
===== TP4/generated/irish_reel_large_T2.0.abc =====
X:1
T:Irish Reel
M:4/4
L:1/8
K:D
|: D,3/2 E, D>G, D2 | G,/8F,/ D, C>"A""F" B, A,"G#m" A, G,"C[EB,]3/4"A" |!b+
 E |[M:6,] .142 x/!c85- [FBc,] |
[M:3/2] (uE11 A,3) !breath!f+ ::!pf!2B |:"^Rusb6d!4) verian" F/D | W5 d/ d/4
E/F/ FB/A/ c/>a/c/<g/<b c/ | d3/2e/4a/8b'- a<fb<D A2 |: d4-" | dA/d<de/A |
 d/<e/{6ec5(A/B/c/A/ |{fa705 f)8- f>a^
[controller] mboukraa@arcadia-slurm-controller:~/TP-HPC (main)
$
```

Les courbes Tensorboard:



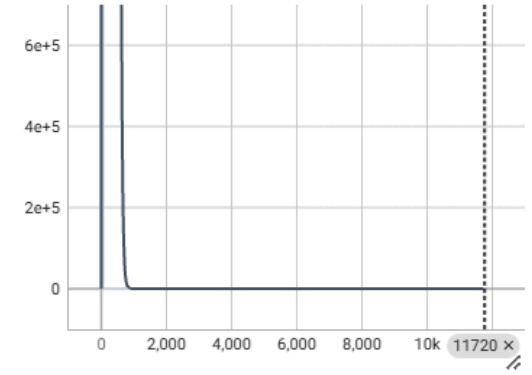
GradNorm 2 cards

GradNorm/Epoch_Avg



Run ↑	Smoothed	Value	Step	Relative
.	0.2813	0.5685	30	25.73 min

GradNorm/step



Run ↑	Smoothed	Value	Step	Relative
.	0.0613	0.0542	11,720	26.54 min